B17 Testing and performance

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B17 Testing and performance 3
B17-L1 Unit tests for data code that catch real bugs 3
B17-L2 Property-based tests with Hypothesis 7
B17-L3 Measure before you optimise 11
Track B best-practice checklist 16
Appendix: solutions and answer keys 17

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B17 Testing and performance

Roadmap: Prerequisite for all months. Unit tests, Hypothesis, profiling and memory use.

B17-L1 Unit tests for data code that catch real bugs

Concept

A test is worth writing if it fails when the code is wrong. For data code, most bugs sit at the edges: empty input, missing values, zero denominators, one row, duplicate keys. Test those on purpose.

With pytest 9, a test is a function whose name starts with `test_` and which uses plain `assert`:

```
import pytest


def test_share_zero_total_is_none():
    assert share(3, 0) is None


@pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25),
   (0, 5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == expected
```

Habits that make tests catch bugs:

- **Test pure functions** (lesson B2-L1). No files, clocks or databases, so tests are fast and exact.
- **Build tiny, explicit inputs** in the test itself: three rows, written out. A reader sees the input and the expected answer together.
- **One behaviour per test**, named for it: `test_zero_total_returns_none`, not `test_share_2`.
- **Compare DataFrames with** `pd.testing.assert_frame_equal`. It checks values, dtypes, column order and index, and prints a clear difference.
- **Compare floats with** `pytest.approx` when the value comes from arithmetic: `assert mean == pytest.approx(0.3)`.
- **Use** `tmp_path` for anything that writes files. pytest gives each test a fresh temporary directory.

Check that each test can fail. Break the code on purpose, once, and watch the test go red. A test that stays green whatever you do protects nothing.

This course's own quality gate does the same: every seeded bug must fail its checker.

Run the tests in CI on every push (lesson A10-L3), and fix a failing test before merging anything else.

Why it matters

A share function had a test, `assert share(1, 4) == 0.25`, and 100% line coverage. The zero-total branch returned `0.0` instead of `None`. Dashboards showed empty segments as "0% of respondents", which the client read as a real finding. A four-word test, `assert share(3, 0) is None`, would have failed on the first run.

Worked example: weak code

```
1 def test_share():
2     result = share(1, 4)
1     assert result
```

1. **Line 3.** Passes for any truthy value: 0.25, 99 or 'oops'. And 0.0 would fail a correct result.

Worked example: strong code

```
1  import pytest
2
3
4  def share(part: float, total: float) -> float | None:
5      """part / total, or None when total is 0."""
6      if total == 0:
7          return None
8      return part / total
9
10
1  @pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25),
       (0, 5, 0.0), (5, 5, 1.0)])
12 def test_share_values(part: float, total: float, expected: float) -> None:
13     assert share(part, total) == pytest.approx(expected)
14
15
16 def test_share_zero_total_is_none() -> None:
2      assert share(3, 0) is None
```

1. **Line 11.** Several cases, including 0 and 1, in one readable table.
2. **Line 17.** The edge case that most often ships broken, tested explicitly.

Test edges on purpose, with tiny explicit inputs, one behaviour per test, and prove each test can fail.

Recap card

- **A test worth writing** One that fails when the code is wrong. Break the code once to prove it.
- **Edge cases for data code** Empty input, missing values, zero denominators, one row, duplicate keys.
- **Compare DataFrames in tests** pd.testing.assert_frame_equal: values, dtypes, columns and index.
- **Compare floats in tests** pytest.approx(expected).

Practice

Exercise 1. Multiple choice (level 1)

Which assertion best tests that `share(3, 0)` returns `None`?

- a) `assert not share(3, 0)`
- b) `assert share(3, 0) is None`
- c) `assert share(3, 0) == 0`
- d) `assert share(3, 0) != 1`

Why?

- r1) It fails for every result except None, including 0.0.
- r2) `is` is faster than `==`.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

```
import pandas as pd

a = pd.DataFrame({"n": [1, 2]})
b = pd.DataFrame({"n": [1.0, 2.0]})
print(a.equals(b), (a == b).all().all())
try:
    pd.testing.assert_frame_equal(a, b)
    print("equal")
except AssertionError:
    print("AssertionError")
```

Exercise 3. Fill the gap (level 2)

Run one test over three cases.

```
@pytest.mark._____(1)(("part", "total", "expected"), [(1, 4, 0.25), (0,
    5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == pytest._____(2)(expected)
```

Exercise 4. Write the function (level 3)

`share(part, total)` should return `part / total`, or `None` when `total` is 0. Write two tests:

- `test_share_basic()`: `share(1, 4)` equals 0.25.
- `test_share_zero_total()`: `share(3, 0)` is `None`.

The checker runs your tests against a correct `share`, which must pass, and against two broken versions, which your tests must catch.

Your code must pass these checks:

- Your tests must pass for a correct share function.
- test_share_zero_total must fail when share returns 0.0 for a zero total. Use `is None`.
- test_share_basic must check the value 0.25, not just that a result exists.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# part / total, or None when total is 0.
def share(part: float, total: float) -> float | None: ...

def test_share_basic() -> None: ...

def test_share_zero_total() -> None: ...
```

Starter code:

```python
def share(part: float, total: float) -> float | None:
    """part / total, or None when total is 0."""
    if total == 0:
        return None
    return part / total


def test_share_basic():
    assert share(1, 4)


def test_share_zero_total():
    assert not share(3, 0)
```

Exercise 5. Architecture design (level 4)

Choose test cases for `weighted_share(df, value, weight)` from lesson B8-L1.

Options: Four rows written out, expected value computed by hand, A 10,000-row sample of production data, expected value from the function itself.

1. Normal case
2. Edge cases

3. Float comparison

Sources

- DataFrame.equals returns False and assert_frame_equal raises for int64 vs float64 columns with equal values, while == reports True. (verified: Run on pandas 3.0.6 in this repository.)

B17-L2 Property-based tests with Hypothesis

Concept

Example tests check the cases you thought of. **Property-based tests** check a rule against hundreds of generated inputs, including ones you did not think of. **Hypothesis** generates the inputs, and when a test fails it **shrinks** the input to the smallest failing case. Examples use Hypothesis 6.168.

```python
from hypothesis import given, strategies as st

@given(st.lists(st.tuples(st.floats(-1e6, 1e6), st.floats(0, 10)),
    min_size=1))
def test_weighted_mean_within_range(pairs):
    values = [v for v, _ in pairs]
    weights = [w for _, w in pairs]
    m = weighted_mean(values, weights)
    assert m is None or min(values) - 1e-6 <= m <= max(values) + 1e-6
```

Run against a `weighted_mean` without a zero guard, this test failed in this course's build with `ZeroDivisionError`, and Hypothesis shrank the input to `pairs=[(0.0, 0.0)]`: one value, one zero weight.

Properties that suit data code:

- **Bounds.** A mean lies between the minimum and maximum. A share lies in [0, 1].
- **Invariance.** Shuffling rows does not change a total. Renaming a category does not change counts.
- **Round trip.** Writing then reading a file gives back the same frame.
- **Idempotence.** Deduplicating twice equals deduplicating once.
- **Grain.** The output has exactly one row per key.
- **Agreement.** A fast implementation equals a slow, obviously correct one.

Choose strategies deliberately. `st.floats()` generates `nan` and infinities by default. Bound it (`st.floats(-1e6, 1e6)`) or pass `allow_nan=False` unless your function must handle them, and then test that behaviour on purpose. `hypothesis.extra.pandas` builds DataFrames with `data_frames` and `column`.

Use `@settings(max_examples=...)` to trade speed for coverage. Hypothesis saves failing examples in a local `.hypothesis/` directory and replays them first on the next run. Add it to `.gitignore`.

Hypothesis does not run in this app's browser runtime. The exercises check your code's structure.

Why it matters

A deduplication step kept the latest record per respondent. Its example tests used three tidy rows. In production, two records for one respondent had the **same** timestamp, and the function kept both, so the respondent was counted twice. A Hypothesis test of the grain, "the output has exactly one row per respondent", fails on this bug at once. In this course's build it shrank the failure to `rows=[('a', 0), ('a', 0)]`: two identical rows.

Worked example: weak code

```
1 @given(st.floats(), st.floats())
2 def test_share_in_range(part, total):
2     assert 0 <= share(part, total) <= 1
```

1. **Line 1.** Unbounded floats include nan, infinities and negatives the function never promised to handle.
2. **Line 3.** Ignores the None branch, so the property is false for a correct function.

Worked example: strong code

```
 1 from hypothesis import given, strategies as st
 2
1 counts = st.integers(min_value=0, max_value=10_000)
 4
 5
 6 @given(counts, counts)
 7 def test_share_in_unit_interval(a: int, b: int) -> None:
2     part, total = min(a, b), max(a, b)
 9     result = share(part, total)
10     if total == 0:
3         assert result is None
12     else:
13         assert result is not None and 0.0 <= result <= 1.0
```

1. **Line 3.** Inputs the function promises to handle: non-negative counts.
2. **Line 8.** Builds valid inputs (part <= total) instead of filtering most examples away.
3. **Line 11.** The zero branch is part of the property.

State a rule that must always hold, generate inputs from the domain the function promises, and let Hypothesis find and shrink the counterexample.

Recap card

- **Property-based test** A rule checked against many generated inputs, with failing inputs shrunk to a minimal case.
- **Four data-code properties** Bounds, invariance, round trip, idempotence, grain, and agreement with a slow version.
- **st.floats() default** Includes nan and infinities. Bound it or pass allow_nan=False on purpose.
- **Hypothesis example database** .hypothesis/ replays past failures. Keep it out of Git.

Practice

Exercise 1. Multiple choice (level 1)

A Hypothesis test fails. What does Hypothesis report?

- a) The first random input that failed.
- b) A shrunk input: the simplest example it found that still fails.
- c) Every failing input it tried.
- d) Only the exception, without the input.

Why?

- r1) Shrinking turns a messy random failure into a minimal case you can read.
- r2) Hypothesis only ever generates simple inputs.

Exercise 2. Predict the output (level 2)

This property test runs with Hypothesis 6.168. Does it pass or fail?

```
from hypothesis import given, strategies as st


@given(st.floats())
def test_equal_to_itself(x):
    assert x == x
```

Exercise 3. Fill the gap (level 2)

Generate bounded floats without nan, and run 500 examples.

```
@_____(1)(max_examples=500)
@given(st.floats(min_value=-1e6, max_value=1e6, _____(2)=False))
def test_round_trip(x):
    assert parse(format(x)) == x
```

Exercise 4. Write the function (level 3)

Write a Hypothesis test `test_dedupe_idempotent` for `dedupe(rows: list[tuple[str, int]]) -> list[tuple[str, int]]`. Generate `rows` as lists of `(id, timestamp)` tuples with `st.lists(st.tuples(st.sampled_from(["a", "b", "c"]), st.integers(0, 5)))`. Assert that `dedupe(dedupe(rows)) == dedupe(rows)`.

Hypothesis does not run in the browser. The checker reads your code. Run it locally with pytest.

What you need to know.

Hypothesis builds test inputs from **strategies**:

- `st.sampled_from(["x", "y"])` picks one value from the list.
- `st.integers(0, 5)` picks an integer from 0 to 5.
- `st.tuples(a, b)` builds a tuple from two strategies, and `st.lists(s)` a list of any length.

`@given(strategy)` above a test function runs it many times. Each run passes a new generated value as the function's argument:

```
@given(st.lists(st.integers(0, 9)))
def test_sort_keeps_length(values: list[int]) -> None:
    assert len(sorted(values)) == len(values)
```

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
def test_dedupe_idempotent(
    rows: list[tuple[str, int]],
) -> None: ...
```

Starter code:

```
from hypothesis import given, strategies as st


def test_dedupe_idempotent():
    rows = [("a", 1), ("a", 1)]
    assert dedupe(rows) == [("a", 1)]
```

Exercise 5. Architecture design (level 4)

Pick a property for each function.

Options: Shuffling rows does not change the result, Write then read returns the same frame, Result lies between the minimum and maximum input, Equals a slow, obviously correct version.

1. weighted_total(df): sum of value × weight

2. save_parquet and load_parquet
3. weighted_mean(values, weights) with positive weights
4. A vectorised rewrite of a row-by-row scoring loop

Sources

- A grain property found a dedupe that keeps timestamp ties and shrank it to rows=[('a', 0), ('a', 0)]. (verified: Run with Hypothesis 6.168.1 and pytest in this repository.)
- Hypothesis 6.168 found a ZeroDivisionError in an unguarded weighted mean and shrank it to pairs=[(0.0, 0.0)]; st.floats() generated nan and failed x == x. (verified: Run with Hypothesis 6.168.1 and pytest in this repository.)

B17-L3 Measure before you optimise

Concept

Most slow data code spends its time in one or two places, and they are rarely where you guess. Measure first, change one thing, measure again. All tools here are in the Python standard library.

Profile to find where time goes. `cProfile` counts calls and time per function:

```
import cProfile, pstats
cProfile.run("main()", "profile.out")
pstats.Stats("profile.out").sort_stats("cumulative").print_stats(10)
```

Sort by `cumulative` to see which of your functions, including everything they call, takes longest. In this course's build, a job calling a pure-Python sum of squares and a NumPy version over 200,000 numbers showed 0.037 s cumulative for the Python one and 0.002 s for NumPy. In a notebook, `%prun main()` does the same.

Time a candidate change with `timeit`. `timeit.repeat(fn, repeat=5, number=100)` returns five totals, each for 100 calls. Report the **minimum** divided by `number`: other processes can only slow a run down, so the fastest run is the best estimate of the code's own cost. Use `time.perf_counter()`, not `time.time()`, for manual timing.

Measure memory when that is the limit. `tracemalloc` reports current and peak allocation: one million float64 values showed a peak of 8.0 MB, as

expected at 8 bytes each. For pandas, `df.memory_usage(deep=True)` counts the contents of text columns, not just the pointers.

Rules:

- Profile with realistic data sizes. A 100-row sample hides what dominates at 10 million rows.
- Keep setup out of the timed region: reading files, building inputs, copying frames.
- Fix the biggest cost first. Speeding up a function that takes 2% of the runtime cannot save more than 2%.
- Keep a test around the code you optimise (lesson B17-L1), so faster does not become wrong.

Why it matters

An analyst spent two days rewriting a weighting function in Numba because "loops are slow in Python". The job still took 40 minutes. A profile, run afterwards, showed 36 of those minutes in reading 300 CSV files one at a time with default type inference. Switching to one Parquet read with explicit dtypes cut the job to 5 minutes. The Numba rewrite saved 20 seconds.

Worked example: weak code

```
start = time.time()
for _ in range(10):
    df = pd.read_csv("big.csv")
    result = transform(df)
print("transform takes", (time.time() - start) / 10)
```

1. **Line 1.** Wall-clock time with low resolution. Use time.perf_counter().
2. **Line 3.** The file read sits inside the timed loop, so the number is mostly I/O.
3. **Line 5.** An average over runs, inflated by any background activity.

Worked example: strong code

```
import time
from collections.abc import Callable


def best_time(fn: Callable[[], object], repeat: int = 5, number: int = 10) -> float:
    """Best per-call time in seconds over `repeat` rounds of `number` calls."""
    if repeat < 1 or number < 1:
        raise ValueError("repeat and number must be at least 1")
    best = float("inf")
    for _ in range(repeat):
```

```
        start = time.perf_counter()
        for _ in range(number):
            fn()
        best = min(best, (time.perf_counter() - start) / number)
    return best
```

1. **Line 11.** A high-resolution clock meant for measuring intervals.
2. **Line 14.** Keep the fastest round: noise only ever adds time.

Profile to find the cost, time the change with perf_counter or timeit, keep setup out of the timing, and fix the biggest cost first.

Recap card

- **Find where time goes** cProfile with pstats sorted by cumulative, or %prun in a notebook.
- **Time a small change** timeit.repeat or perf_counter. Report the minimum per call.
- **Measure memory** tracemalloc peak, and df.memory_usage(deep=True) for pandas.
- **The order of work** Profile, fix the biggest cost, re-measure, keep a test around the change.

Practice

Exercise 1. Multiple choice (level 1)

A job takes 40 minutes. A profile shows 36 minutes in file reading and 1 minute in the weighting function. What should you optimise first?

- a) The weighting function, because loops are slow.
- b) File reading.
- c) Both equally.
- d) Neither: buy a faster machine.

Why?

- r1) A 1-minute function can save at most 1 minute. Reading holds 90% of the time.
- r2) I/O is always the bottleneck.

Exercise 2. Predict the output (level 2)

What does this print?

```
import timeit

rounds = timeit.repeat(lambda: sum(range(1000)), repeat=5, number=100)
```

```
print(len(rounds))
```

Exercise 3. Fill the gap (level 2)

Profile `main()` and print the 10 functions with the highest cumulative time.

```
cProfile.run("main()", "profile.out")
pstats.Stats("profile.out").sort_stats("_____(1)")._____(2)(10)
```

Exercise 4. Write the function (level 3)

Write `best_time(fn, repeat=5, number=10) -> float`. Run `repeat` rounds. In each round, call `fn()` `number` times, timed with `time.perf_counter()`, and compute the per-call time. Return the smallest per-call time. Raise `ValueError` if `repeat` or `number` is below 1.

Your code must pass these checks:

- Call fn number times in each of repeat rounds (3 × 4 = 12 calls), and return a float.
- Return the smallest round time divided by number: rounds of 2, 1 and 3 seconds over 2 calls give 0.5.
- repeat or number below 1 must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Best per-call time over several rounds.
def best_time(
    fn: Callable[[], object],
    repeat: int = 5,
    number: int = 10,
) -> float: ...
```

Starter code:

```
import time


def best_time(fn, repeat=5, number=10):
    start = time.time()
    fn()
    return time.time() - start
```

Exercise 5. Spot the bug (level 4)

This benchmark says `add_features` takes 1.8 seconds. The function itself takes about 0.1 seconds. Tap the line that distorts the measurement.

```
import time

import pandas as pd

```

```

def benchmark(path: str, rounds: int) -> float:
    """Best time of add_features over rounds."""
    best = float("inf")
    for _ in range(rounds):
        start = time.perf_counter()
        df = pd.read_parquet(path)
        add_features(df)
        best = min(best, time.perf_counter() - start)
    return best
```

Write down the line number of the bug. Then write the fix.

Sources

- cProfile showed 0.037 s cumulative for a pure-Python sum of squares over 200,000 numbers and 0.002 s for NumPy; tracemalloc peak for one million float64 values was 8.0 MB. (verified: Run on Python 3.11 with NumPy 2.4.6 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B17-L1 Unit tests for data code that catch real bugs

B17-L1 exercise 1

Hints. (1) Which buggy values would each assertion let through? (2) Is 0.0 falsy? (3) Only one rejects 0.0.

Answer. b) `assert share(3, 0) is None` Because: r1) It fails for every result except None, including 0.0.

Write assertions that pass only for the correct value.

Common wrong answers.

- `a`: 0.0 is falsy too, so a buggy 0.0 would pass.
- `c`: That asserts the bug.
- `d`: Almost any wrong value passes.
- `b|r2`: Right answer, wrong reason. Precision matters, not speed.

B17-L1 exercise 2

Hints. (1) What are the dtypes? (2) Does == compare dtypes? (3) assert_frame_equal does.

Answer.

```
1 False True
2 AssertionError
```

`==` compares values only. `assert_frame_equal` also checks dtypes, which catches a column that silently became float.

Common wrong answers.

- `True True equal`: Values match, but dtypes differ (int64 and float64). assert_frame_equal checks dtypes by default.

B17-L1 exercise 3

Hints. (1) The pytest marker for a table of cases. (2) Float comparison with tolerance. (3) `parametrize` and `approx`.

Answer.

```python
@pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25),
    (0, 5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == pytest.approx(expected)
```

`parametrize` runs the test once per row. `approx` compares floats with a sensible tolerance.

Common wrong answers.

- g1 = `parameterize`: pytest spells it parametrize, without the extra e.

B17-L1 exercise 4

Hints. (1) A test must fail for wrong values, not only for crashes. (2) Compare with the exact expected value: `== 0.25`. (3) Test for None with `is None`.

Answer.

```python
def share(part: float, total: float) -> float | None:
    """part / total, or None when total is 0."""
    if total == 0:
        return None
    return part / total


def test_share_basic() -> None:
    assert share(1, 4) == 0.25


def test_share_zero_total() -> None:
    assert share(3, 0) is None
```

The checker plays the role of a mutation test: it breaks the code and checks your tests notice. Precise assertions catch both bugs.

B17-L1 exercise 5

Hints. (1) Where does the expected value come from? (2) Which inputs break data code? (3) Floating point.

Answer.

- Tiny explicit inputs with hand-computed answers.
- Every edge the function promises to handle.
- approx for floats that come from arithmetic.

Independent expected values, every edge, and tolerant float comparison make tests that catch bugs.

Common wrong answers.

- normal = `sample`: An expected value produced by the function under test only checks that it agrees with itself.
- edges = `skip`: Line coverage is not behaviour coverage. Edge branches are where bugs ship.
- compare = `exact`: 0.1 + 0.2 != 0.3 in floating point. Use approx.

B17-L2 Property-based tests with Hypothesis

B17-L2 exercise 1

Hints. (1) What makes a failure easy to debug? (2) A small input. (3) Shrinking.

Answer. b) A shrunk input: the simplest example it found that still fails. Because: r1) Shrinking turns a messy random failure into a minimal case you can read.

Shrinking reduced a weighted-mean failure to one value with one zero weight.

Common wrong answers.

- `a`: Hypothesis keeps simplifying the failing input before reporting it.
- `c`: It reports one minimal example, not the whole search.
- `d`: The failing example is shown, such as pairs=[(0.0, 0.0)].
- `b|r2`: Right answer, wrong reason. It generates complex inputs, then shrinks the failure.

B17-L2 exercise 2

Hints. (1) What special values can st.floats() produce? (2) Is nan equal to itself? (3) No.

Answer.

```
fail
```

The test fails on `x=nan`. Bound the strategy or pass `allow_nan=False` unless the function must handle nan.

Common wrong answers.

- `pass`: st.floats() generates nan by default, and nan == nan is False.

B17-L2 exercise 3

Hints. (1) The decorator that configures a test. (2) The floats argument for nan. (3) `settings` and `allow_nan`.

Answer.

```
@settings(max_examples=500)
@given(st.floats(min_value=-1e6, max_value=1e6, allow_nan=False))
def test_round_trip(x):
    assert parse(format(x)) == x
```

`@settings(max_examples=500)` raises the search budget. `allow_nan=False` keeps nan out when the function does not promise to handle it.

Common wrong answers.

- g2 = `allow_infinity`: Bounds already exclude infinities. The gap is for nan.

B17-L2 exercise 4

Hints. (1) Decorate with @given and the strategy from the prompt. (2) The test receives rows as its argument. (3) Compare dedupe applied twice with dedupe applied once.

Answer.

```
from hypothesis import given, strategies as st


@given(st.lists(st.tuples(st.sampled_from(["a", "b", "c"]),
   st.integers(0, 5))))
def test_dedupe_idempotent(rows: list[tuple[str, int]]) -> None:
    assert dedupe(dedupe(rows)) == dedupe(rows)
```

A small id alphabet and small timestamps make duplicates and ties likely. Idempotence is one property. Pair it with a grain property, one row per id, which catches a dedupe that keeps ties: that bug is idempotent, so idempotence alone misses it.

B17-L2 exercise 5

Hints. (1) What must never change? (2) What must come back? (3) What can be compared?

Answer.

- Totals do not depend on row order.
- Saving and loading must not lose data or dtypes.
- A weighted mean stays within the range of its inputs.
- A rewrite must agree with the code it replaces.

Good properties come from what the function promises: invariance, round trips, bounds and agreement.

Common wrong answers.

- total = `bounds`: A total can exceed every input value. Order invariance is the property that holds.
- io = `invariance`: For I/O the key promise is that nothing is lost: a round trip.
- fast = `bounds`: Bounds would pass many wrong rewrites. Compare with the trusted version.

B17-L3 Measure before you optimise

B17-L3 exercise 1

Hints. (1) What is the most a speed-up of each part can save? (2) Compare 36 with 1. (3) Follow the profile.

Answer. b) File reading. Because: r1) A 1-minute function can save at most 1 minute. Reading holds 90% of the time.

Fix the biggest measured cost first. Guesses about "slow loops" waste days.

Common wrong answers.

- `a`: Even an infinitely fast weighting function saves only 1 minute of 40.
- `c`: Effort should follow cost. One of them holds 36 of 40 minutes.
- `d`: Hardware is a last resort once the code's own costs are known.
- `b|r2`: Right answer, wrong reason. Here the profile says I/O. Elsewhere it may not.

B17-L3 exercise 2

Hints. (1) What does repeat control? (2) What does number control? (3) One total per round.

Answer.

```
5
```

Five rounds, each the total time for 100 calls. Report `min(rounds) / 100` as the per-call time.

Common wrong answers.

- `100`: number is the calls per round. repeat is the number of rounds returned.

- `500`: repeat returns one total per round, not one per call.

B17-L3 exercise 3

Hints. (1) Time including everything a function calls. (2) The Stats method that prints rows. (3) `cumulative` and `print_stats`.

Answer.

```
cProfile.run("main()", "profile.out")
pstats.Stats("profile.out").sort_stats("cumulative").print_stats(10)
```

Cumulative time points to the high-level function that owns the cost. Then drill into it.

Common wrong answers.

- g1 = `calls`: Sorting by call count shows busy functions, not expensive ones. Sort by cumulative time.

B17-L3 exercise 4

Hints. (1) Validate repeat and number first. (2) Time each round with perf_counter before and after the inner loop. (3) Keep the minimum of round_time / number.

Answer.

```
import time
from collections.abc import Callable


def best_time(fn: Callable[[], object], repeat: int = 5, number: int = 10) -> float:
    """Best per-call time over several rounds.

    Args:
        fn: Function to time, called with no arguments.
        repeat: Number of rounds.
        number: Calls per round.

    Returns:
        The smallest per-call time in seconds.

    Raises:
        ValueError: If repeat or number is below 1.
    """
    if repeat < 1 or number < 1:
        raise ValueError("repeat and number must be at least 1")
    best = float("inf")
    for _ in range(repeat):
        start = time.perf_counter()
        for _ in range(number):
```

```
            fn()
        best = min(best, (time.perf_counter() - start) / number)
    return best
```

The minimum over rounds estimates the code's own cost, because interference only adds time. Dividing by `number` gives a per-call figure you can compare.

B17-L3 exercise 5

Hints. (1) What runs between start and the end of the round? (2) Is reading the file part of add_features? (3) Move setup out of the timed region.

Answer. The bug is on line 11.

Fixed version:

```
import time

import pandas as pd


def benchmark(path: str, rounds: int) -> float:
    """Best time of add_features over rounds."""
    df = pd.read_parquet(path)
    best = float("inf")
    for _ in range(rounds):
        start = time.perf_counter()
        add_features(df)
        best = min(best, time.perf_counter() - start)
    return best
```

The file read is setup, and it dominates the timing. Read once before the loop, then time only the function.

Common wrong answers.

- Line 13: Taking the minimum is right. The timed region includes more than the function.